

SmartNav — Work Split

Hackathon Build · 4 Members · Each owns one layer · Integrate at the end

Member 1 — Maps + Routing Core

Goal: Fully functional map, routing, and drag-to-reroute.

- React + Vite + Tailwind project setup
- React Leaflet map, OpenStreetMap tiles, Bangalore default center
- OpenRouteService API — generate route between two points
- Render route polyline with start/end markers
- Draggable route using Leaflet Routing Machine
- On drag: recalculate route, extract updated waypoints { lat, lng }
- On drag: identify affected segment_ids, write prefer_count++ to Firestore segments
- On route load: read segment scores from Firestore to visually weight routing
- SpeechSynthesis API — speak AI warning text (string passed in from sessions doc)

Deliverable:

- User drags route → new route rendered → prefer_count written to Firestore

Hardcode on Day 0 (replace at integration):

- Firestore write: hardcode a fixed segment_id with static prefer_count++ call
 - Firestore read: hardcode dummy segment score array for route weighting
 - SpeechSynthesis: hardcode a static warning string to test voice
-

Member 2 — Firebase + Realtime + Heatmap + UI

Goal: All collaboration, realtime sync, and UI panels.

- Firebase project setup — Firestore + Auth + env config
- Define and own all 3 Firestore collections: segments, sessions, reports
- Session system — host creates session doc, guest joins via session_id
- WhatsApp deep link — share session: wa.me/?text=Join+my+SmartNav+session+{session_id}
- onSnapshot on sessions doc → push new live_waypoints to guest map (calls M1's route renderer)
- onSnapshot on segments collection → re-render heatmap on prefer/avoid count change
- Heatmap overlay — green (preferred), red (avoided), pulse animation for reports < 5 min old
- Report modal — user types text → writes to reports collection (report_text, segment_id, created_at)
- Validation log UI — reads ai_category, ai_severity, verification_status from reports doc
- Warning card UI — reads ai_warning field from sessions doc, displays on map load
- WhatsApp notify popup — reads delay_flag + delay_message from sessions doc → shows popup with wa.me link
- 'Why locals avoid this' panel — reads why_avoid_text from sessions doc, shows on segment click
- Mobile responsiveness across all panels

Deliverable:

- Host drags route → guest sees update instantly → heatmap pulses → all UI panels populate from Firestore

Hardcode on Day 0 (replace at integration):

- sessions doc: hardcode static live_waypoints array to test guest sync
 - reports doc: hardcode static ai_category + verification_status to test validation log UI
 - sessions doc: hardcode static ai_warning string to test warning card
 - sessions doc: hardcode delay_flag: true + static message to test notify popup
-

Member 3 — AI + Intelligence Layer

Goal: All AI calls. Writes structured output to Firestore. M2 reads and displays.

- Reporter Agent — listens for new reports docs (report_text set, ai_category empty)
 - Calls AI API: report_text → { type, severity, location }
 - Writes ai_category + ai_severity back to that reports doc
- Verification Agent — after Reporter writes, reads report + weather API + recent reports + timestamps
 - Assigns verification_status: verified / low_confidence / suspicious
 - Writes verification_status to reports doc
- Weather API integration — fetch rain/weather for Bangalore
- Route Intelligence — on session create, reads segment avoid_counts + current time
 - AI generates contextual warning string
 - Writes ai_warning to sessions doc (M2 reads and shows warning card)
- 'Why locals avoid this' — reads recent report_texts for a segment → AI summary string
 - Writes why_avoid_text to sessions doc (M2 reads on segment click)
- Predictive risk — rain from weather API + historical flooding reports → risk string included in ai_warning
- Proactive delay detection — reads route segments for severe congestion (avoid_count threshold)
 - Writes delay_flag: true + delay_message to sessions doc (M2 reads and shows popup)
- Confidence scoring — high (reports > 10), medium (reports > 3), low otherwise
 - Writes confidence field to segments doc
- Prompt engineering, JSON parsing, retry handling for all AI calls

Deliverable:

- User submits report → Reporter structures it → Verification Agent validates it → all fields written to Firestore → M2 UI updates automatically

Hardcode on Day 0 (replace at integration):

- Read from reports: hardcode static report_text to test Reporter Agent
- Read from sessions: hardcode static session_id + route to test Route Intelligence
- Weather API: hardcode static weather response to test Verification + Predictive logic

Member 4 — PPT + Demo + QA + Coordination

Goal: Judges see a polished, stable, impressive demo.

- PPT — architecture diagram showing all 4 layers and how they connect
- PPT — feature slides matching all 10 features from idea doc
- Judge pitch line: 'We built a collaborative navigation layer where local residents actively improve routes. Our dual-agent AI structures and verifies noisy reports before updating shared traffic intelligence.'
- 60-second demo script — timed exactly to idea doc demo flow:
 - 0–10s: enter route, heatmap visible, AI warning appears
 - 10–30s: share session, host reroutes, guest updates instantly
 - 30–45s: report flooding, agents validate, heatmap pulses red
 - 45–55s: AI asks to notify contact, WhatsApp message generated
 - 55–60s: new user sees community preferred route + AI explanation
- Seed demo data in Firestore:
 - segments: Domlur (avoid_count: 14, prefer_count: 3), Sony Signal (flooding reports)
 - sessions: one active host-guest session
 - reports: 3 pre-seeded reports with verification_status set
- Backup demo video recording
- End-to-end QA — test all features against demo script
- Git coordination — own main branch, manage PRs, handle merges

Deliverable:

- Hackathon-ready PPT, seeded Firestore, stable demo, backup video
-

Firestore Contracts — Lock on Day 0

These are the exact field names everyone must use. Do not rename fields at integration.

segments collection

Owner: M2 creates. M1 writes prefer/avoid. M3 writes confidence. M2 reads for heatmap.

```
{ segment_id, prefer_count, avoid_count, confidence, last_updated }
```

sessions collection

Owner: M2 creates. M1 writes live_waypoints. M3 writes ai_warning, delay_flag, delay_message, why_avoid_text. M2 reads all.

```
{ session_id, guest_route, live_waypoints, route_geometry, status,
  ai_warning, delay_flag, delay_message, why_avoid_text }
```

reports collection

Owner: M2 creates + writes report_text. M3 writes ai_category, ai_severity, verification_status. M2 reads for validation log.

```
{ segment_id, report_text, ai_category, ai_severity, verification_status, created_at
}
```

AI Output Shapes (M3 internal — must match these exactly)

Reporter Agent output:

```
{ "type": "flooding", "severity": "high", "location": "Sony Signal" }
```

Verification Agent output:

```
{ "verified": true, "confidence": "high" }
```

Route Intelligence output (written to sessions.ai_warning):

```
{ "warning_text": "Heavy school traffic likely near Domlur between 8-10 AM." }
```

Delay detection output (written to sessions):

```
{ "delay_flag": true, "delay_minutes": 15, "delay_message": "Running 15 min late due
to severe congestion near Domlur flyover." }
```

Integration — How to Merge

- M2 already has Firestore live — everyone else just replaces hardcoded dicts with real Firestore reads/writes
 - M1: replace hardcoded segment write with real segment_id extracted from drag
 - M3: replace hardcoded report_text with real onSnapshot listener on reports collection
 - M3: replace hardcoded session_id with real session doc read for Route Intelligence
 - M2 UI: all panels already listening via onSnapshot — no changes needed if field names match contracts
 - Rule: if integration breaks, a field name is wrong — fix the contract, not the logic
-

Absolute Priority**MUST:**

- Host drags route → guest route updates instantly

SECOND:

- Heatmap visibly changes on new reports

THIRD:

- AI validates and structures reports

Everything else is bonus.